

““latex
““

Міністерство освіти і науки України
Маріупольський державний університет

Кафедра системного аналізу та інформаційних технологій

Лабораторна робота №12–13

з навчальної дисципліни «Моделі розподілених баз даних»

Розподілені транзакції у розподілених базах даних.
Практична реалізація двофазного протоколу фіксації (2PC) у
PostgreSQL

Варіант 3

Виконав студент:

Василенко Андрій Васильович

IV курсу

денної форми навчання

спеціальності 124 (СА)

Перевірила:

доцент каф. САІТ

к.т.н., доц. Марченко Н. Б.

Київ – 2026

Зміст

МЕТА РОБОТИ	3
1 ТЕОРЕТИЧНІ ОСНОВИ ТА УМОВИ МОДЕЛЮВАННЯ	3
2 МОДЕЛЬ БАНКІВСЬКОГО ПЕРЕКАЗУ ТА ОБРОБЛЕННЯ ВІДМОВ	6
3 ПРАКТИЧНА РЕАЛІЗАЦІЯ ТА ОЦІНЮВАННЯ РЕЗУЛЬТА- ТІВ У POSTGRESQL	9
ВИСНОВКИ	12
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ	13
ДОДАТОК А	14

МЕТА РОБОТИ

Метою лабораторної роботи є дослідження принципів керування транзакціями у розподіленому середовищі та практична реалізація банківського переказу між двома вузлами бази даних PostgreSQL із застосуванням двофазного протоколу фіксації (2PC).

Для досягнення поставленої мети необхідно виконати такі завдання:

- засвоїти принципи керування транзакціями у розподіленому середовищі;
- розглянути механізми забезпечення узгодженості даних;
- порівняти протоколи фіксації транзакцій;
- оцінити ризики паралельного доступу та відмов вузлів;
- реалізувати модель переказу коштів між двома вузлами PostgreSQL із використанням 2PC.

1 ТЕОРЕТИЧНІ ОСНОВИ ТА УМОВИ МОДЕЛЮВАННЯ

Постановка задачі.

Необхідно дослідити виконання розподіленої транзакції на прикладі банківського переказу між рахунками, які розміщені на різних вузлах. У практичній моделі Node A зберігає рахунок відправника, Node B — рахунок отримувача, а координатор приймає єдине глобальне рішення: COMMIT або ROLLBACK. Часткове виконання операції є неприпустимим, оскільки списання коштів без відповідного зарахування порушує цілісність даних.

Два вузли моделюються окремими базами даних PostgreSQL: bank_node_a та bank_node_b. Координатор реалізовано як Python-програму, що встановлює підключення до обох баз і використовує команди PREPARE TRANSACTION, COMMIT PREPARED та ROLLBACK PREPARED. У реальній розподіленій системі ці бази можуть функціонувати на різних серверах.

Для використання prepared transactions параметр PostgreSQL max_prepared_transactions має бути більшим за нуль. Підготовлені транзакції не потрібно залишати незавершеними протягом тривалого часу, оскільки вони можуть утримувати блокування та зменшувати доступність

ресурсів.

Порівняння протоколів фіксації.

Основні характеристики підходів до завершення розподілених транзакцій наведено в табл. 1.1.

Таблиця 1.1 — Порівняльний аналіз протоколів фіксації транзакцій

Протокол	Принцип роботи	Переваги	Недоліки	Рівень надійності
Двофазний протокол (2PC)	Фаза PREPARE: усі вузли перевіряють операцію та голосують READY/ABORT. Після цього координатор надсилає COMMIT або ROLLBACK.	Забезпечує атомарне спільне рішення; підтримується PostgreSQL.	Після READY учасник утримує ресурси до рішення координатора; збій координатора знижує доступність.	Високий щодо атомарності; залежить від відновлення координатора.
Трифазний протокол (3PC)	Містить проміжну фазу PRE-COMMIT між голосуванням та остаточною фіксацією.	Зменшує ризик тривалого блокування за визначених припущень щодо тайм-аутів і мережі.	Потребує більшої кількості повідомлень; не усуває всі проблеми поділу мережі.	Високий у визначеній моделі відмов.
Оптимістичні методи	Операції виконуються без тривалих блокувань, а перед фіксацією виконується перевірка конфліктів.	Забезпечують добру продуктивність за малої кількості конфліктів.	За конкуренції спричиняють скасування та повторні спроби; розподілена валідація є складнішою.	Залежить від алгоритму валідації та повторення операцій.

Властивості транзакцій у розподіленому середовищі.

Застосування 2PC пов'язане із забезпеченням властивостей ACID. Способи їх підтримання в обраній моделі подано в табл. 1.2.

Таблиця 1.2 — Забезпечення властивостей ACID у розподіленій системі

Властивість	Зміст	Спосіб забезпечення у моделі
Атомарність	Зміни виконуються на всіх вузлах або не виконуються ніде.	Координатор 2PC; PREPARE TRANSACTION на кожному вузлі; єдине рішення COMMIT PREPARED або ROLLBACK PREPARED.
Узгодженість	Після завершення транзакції дані відповідають правилам предметної області.	Обмеження CHECK, PRIMARY KEY, перевірка активності рахунку та достатності коштів до стану READY.
Ізольованість	Паралельні операції не повинні формувати некоректний результат.	Рівень SERIALIZABLE, блокування змінюваних рядків і механізм MVCC PostgreSQL.
Довговічність	Зафіксовані зміни зберігаються після збою.	Журнал WAL та збережений стан prepared transaction; після відновлення координатор завершує зафіксоване рішення.

Атомарність складніше забезпечити у розподіленій системі, ніж у централізованій, оскільки одна логічна операція змінює дані на кількох незале-

жних вузлах. Один із вузлів може виконати локальну зміну, а інший — втратити зв'язок або завершити оброблення з помилкою. Саме тому необхідний координатор та протокол, який унеможливило фіксацію лише частини переказу.

Взаємне блокування може виникнути, якщо паралельні транзакції захоплюють ресурси в різному порядку: перша транзакція утримує рахунок А й очікує рахунок В, а друга утримує рахунок В та очікує рахунок А. У 2PC значення цього ризику зростає, оскільки `prepared transaction` продовжує утримувати блокування до отримання глобального рішення.

Критерії вибору механізму завершення транзакції.

Вибір протоколу завершення розподіленої транзакції визначається не лише вимогою атомарності, а й допустимим часом блокування ресурсів, імовірністю відмов та можливістю відновити рішення координатора. Для операції переказу коштів пріоритетною є неможливість часткової фіксації: кінцевий стан має містити одночасно зменшення балансу відправника і збільшення балансу отримувача. Тому в моделі доцільно застосовувати 2PC, навіть якщо він передбачає очікування остаточного рішення після фази підготовки.

На фазі `PREPARE` кожний учасник перевіряє локальні обмеження: існування рахунку, його активність та достатність залишку на рахунку відправника. Після успішної перевірки вузол зберігає підготовлений стан у журналі та повідомляє координатору `READY`. З цього моменту локальна зміна ще не доступна як остаточно зафіксований результат, однак необхідні для неї ресурси можуть залишатися заблокованими до надходження команди завершення.

З практичної точки зору надійність 2PC потребує контролю за тривалістю `prepared transactions`. Координатор повинен зберігати глобальне рішення до його підтвердженого виконання кожним вузлом, а адміністратор — мати можливість перевірити незавершені операції через системне подання `pg_prepared_xacts`. Таке поєднання протоколу, журналювання та діагностики забезпечує відновлюваність операції без порушення узгодженості даних.

2 МОДЕЛЬ БАНКІВСЬКОГО ПЕРЕКАЗУ ТА ОБРОБЛЕННЯ ВІДМОВ

Послідовність виконання транзакції.

У моделі виконується переказ суми 250.00 UAH із рахунку 101, розміщеного на Node A, до рахунку 202, розміщеного на Node B. Процес оброблення операції складається з таких кроків:

1. Координатор створює глобальну операцію переказу 250.00 UAH.
2. Node A відкриває транзакцію, перевіряє активність рахунку та достатність залишку, виконує попереднє списання й переходить до стану PREPARED.
3. Node B відкриває транзакцію, перевіряє активність рахунку отримувача, виконує попереднє зарахування й переходить до стану PREPARED.
4. Якщо обидва вузли відповіли READY, координатор записує рішення COMMIT та виконує COMMIT PREPARED на обох вузлах.
5. Якщо хоча б один учасник не готовий до фіксації, координатор приймає рішення ROLLBACK і скасовує вже підготовлені зміни.

Обґрунтування вибору протоколу.

Для задачі використано двофазний протокол фіксації (2PC), оскільки банківський переказ не допускає часткового результату: списання без зарахування або зарахування без списання. PostgreSQL надає команди PREPARE TRANSACTION, COMMIT PREPARED та ROLLBACK PREPARED, які дають змогу змодельовати дві фази завершення транзакції.

Роль координатора полягає у створенні ідентифікатора глобальної операції, надсиланні запиту на підготовку вузлам, збиранні відповідей, збереженні остаточного рішення й передаванні його кожному учаснику. Якщо після запису рішення відбувається збій, координатор після відновлення повторює завершальну команду відповідно до журналу рішень.

Вплив мережевих затримок і відмов.

Мережеві затримки збільшують час очікування повідомлень READY/ABORT та період утримання блокувань. Якщо рішення COMMIT ще не було зафіксовано, відсутність відповіді від учасника може призвести до рішення про відкат.

Якщо ж рішення COMMIT уже записане, його необхідно повторно передати вузлам після відновлення з'єднання. Можливі ситуації відмов і способи реагування наведено в табл. 2.1.

Таблиця 2.1 — Можливі збої та механізми відновлення

Збій	Наслідок	Механізм відновлення
Недостатньо коштів на Node A	Node A повертає ABORT; зарахування не має бути зафіксоване.	Координатор надсилає ROLLBACK усім підготовленим учасникам.
Неактивний рахунок на Node B	Node B не може прийняти кошти.	Для Node A, якщо він уже перебуває у стані PREPARED, виконується ROLLBACK PREPARED.
Розрив мережі до рішення COMMIT	Координатор не отримує всі голоси.	Координатор фіксує ROLLBACK і після відновлення з'єднання скасовує prepared transactions.
Збій після запису рішення COMMIT	Один із вузлів може залишитися у стані PREPARED.	Після відновлення координатор читає журнал рішення та повторює COMMIT PREPARED.
Збій координатора після PREPARE	Блокування можуть утримуватися протягом тривалого часу.	Відновлений координатор перевіряє pg_prepared_xacts і завершує операцію відповідно до журналу рішення.

Отже, обрана модель дає змогу розмежувати локальне виконання операцій на вузлах і глобальне рішення координатора. Це забезпечує узгоджене завершення переказу навіть за наявності відмов, для яких у журналі координатора збережено остаточний стан операції.

Алгоритм ухвалення глобального рішення.

Життєвий цикл переказу можна подати як послідовність станів глобальної операції. Спочатку координатор реєструє ідентифікатор транзакції й відкриває локальні операції на обох вузлах. Після виконання перевірок він отримує від учасників голоси. Якщо отримано два повідомлення READY, у журналі координатора фіксується рішення COMMIT; лише після цього команда передається вузлам. Якщо на будь-якому етапі до фіксації рішення отримано ABORT або вичерпано припустимий час очікування, координатор зберігає рішення ROLLBACK.

Виконання завершальної фази має бути ідемпотентним з погляду координатора: після відновлення він повторно перевіряє наявність підготовлених транзакцій і надсилає те саме збережене рішення. Координатор не повинен самостійно замінювати раніше записаний COMMIT на ROLLBACK, оскільки це створило б відмінні результати на вузлах. Саме незмінність глобального рішення

Таблиця 2.2 — Стани глобальної операції переказу

Стан	Дія координатора	Очікуваний результат
STARTED	Створення глобального ідентифікатора та відкриття локальних транзакцій.	Обидва вузли готові виконати перевірки й локальні зміни.
PREPARING	Надсилання запиту підготовки та збирання відповідей.	Кожний учасник повертає READY або ABORT.
DECIDED	Запис рішення COMMIT або ROLLBACK до журналу координатора.	Рішення може бути відновлене після збою координатора.
FINISHED	Надсилання завершальної команди всім учасникам.	Баланси зафіксовано узгоджено або повернуто до попередніх значень.

є умовою коректного відновлення розподіленої транзакції.

3 ПРАКТИЧНА РЕАЛІЗАЦІЯ ТА ОЦІНЮВАННЯ РЕЗУЛЬТАТІВ У POSTGRESQL

Структура програмної моделі.

Практична реалізація містить два вузли даних і програму-координатор. Склад компонентів наведено в табл. 3.1.

Таблиця 3.1 — Структура компонентів практичної реалізації

Компонент	База даних або засіб	Призначення
Node A	bank_node_a	Зберігає таблицю <code>accounts</code> із рахунком відправника 101 і таблицю <code>transfer_journal</code> .
Node B	bank_node_b	Зберігає таблицю <code>accounts</code> із рахунком отримувача 202 і таблицю <code>transfer_journal</code> .
Координатор	Python-програма	Збирає відповіді <code>READY/ABORT</code> , записує рішення та завершує <code>prepared transactions</code> .

Підготовка середовища PostgreSQL.

Команди запускаються у `psql` від імені користувача, який має право створювати бази даних та змінювати параметри сервера. Якщо значення `max_prepared_transactions` дорівнює 0, функціональність `prepared transactions` потрібно активувати, після чого перезапустити службу PostgreSQL.

Лістинг 3.1 — Перевірка та налаштування `prepared transactions`

```
1 SHOW max_prepared_transactions;
2
3 -- Якщо результат дорівнює 0:
4 ALTER SYSTEM SET max_prepared_transactions = 10;
5
6 -- Після перезапуску служби PostgreSQL:
7 \i 01_setup_nodes.sql
```

Виконання успішного переказу.

Успішний сценарій зберігається у файлі `02_successful_transfer_2pc.sql`. На першій фазі обидва вузли виконують локальні зміни та готують транзакції, а після отримання двох відповідей `READY` координатор виконує глобальну фіксацію.

Лістинг 3.2 — Успішний сценарій переказу за протоколом 2PC

```
1 -- Node A: попереднє списання
2 BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

```

3 UPDATE accounts SET balance = balance - 250.00
4 WHERE account_id = 101 AND is_active = true AND balance >= 250.00;
5 PREPARE TRANSACTION 'transfer_001_node_a';
6
7 -- Node B: попереднє зарахування
8 BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
9 UPDATE accounts SET balance = balance + 250.00
10 WHERE account_id = 202 AND is_active = true;
11 PREPARE TRANSACTION 'transfer_001_node_b';
12
13 -- Координатор після двох відповідей READY
14 COMMIT PREPARED 'transfer_001_node_a';
15 COMMIT PREPARED 'transfer_001_node_b';

```

Після успішної фіксації очікувані значення балансів подано в табл. 3.2. Одиницю грошового вимірювання позначено міжнародним кодом UAH.

Таблиця 3.2 — Очікувані результати успішного переказу

Вузол і рахунок	Початковий баланс, UAH	Операція, UAH	Кінцевий баланс, UAH
Node A, рахунок 101	1000.00	−250.00	750.00
Node B, рахунок 202	300.00	+250.00	550.00

До виконання остаточного COMMIT стан підготовлених транзакцій можна перевірити запитом, наведеним у лістингу 3.3.

Лістинг 3.3 — Перегляд підготовлених транзакцій

```

1 SELECT gid, database, owner, prepared
2 FROM pg_prepared_xacts
3 WHERE gid IN ('transfer_001_node_a', 'transfer_001_node_b');

```

Виконання сценарію відкату.

У файлі 03_abort_transfer_2pc.sql моделюється ситуація, коли рахунок отримувача стає неактивним. Якщо Node A уже підготував списання, а Node B повернув ABORT, координатор скасовує підготовлену транзакцію відправника. У цьому випадку баланс рахунку 101 залишається на рівні 1000.00 UAH.

Лістинг 3.4 — Сценарій відкату підготовленої транзакції

```

1 -- Node B не готовий прийняти переказ
2 UPDATE accounts SET is_active = false WHERE account_id = 202;
3
4 -- Node A уже підготував списання
5 PREPARE TRANSACTION 'transfer_002_node_a';
6
7 -- Рішення координатора
8 ROLLBACK PREPARED 'transfer_002_node_a';

```

Узагальнення результатів реалізації.

Склад допоміжних файлів, команди запуску програмного координатора та перелік ілюстрацій для підтвердження результатів наведено в додатку А. Основна частина експерименту підтверджує: за успішного сценарію змінюються обидва баланси, а за відмови підготовлена операція скасовується без часткового переказу.

ВИСНОВКИ

У ході лабораторної роботи досліджено особливості розподілених транзакцій і властивості ACID у середовищі з кількома вузлами. Порівняльний аналіз показав, що двофазний протокол фіксації є придатним для моделювання банківського переказу, оскільки забезпечує єдине рішення для всіх учасників операції. Водночас його використання потребує контролю часу перебування транзакцій у стані PREPARED, оскільки підготовлені операції можуть утримувати блокування.

Для PostgreSQL сформовано модель із двох баз даних і координатора. Успішний сценарій завершує зміну балансів через COMMIT PREPARED: сума 250.00 UAH списується з рахунку відправника та зараховується на рахунок отримувача. У сценарії відмови команда ROLLBACK PREPARED скасовує підготовлені зміни, унаслідок чого часткове виконання переказу не виникає.

Застосування журналу рішень координатора дає змогу відновити завершення транзакції після збою зв'язку або перезапуску програми. Отже, поставлену мету досягнуто: принципи 2PC проаналізовано, модель розподіленого переказу побудовано, а очікувані результати успішної фіксації та відкату визначено.

Література

- [1] Методичні вказівки: «Лабораторна робота №12, 13. Розподілені транзакції у розподілених базах даних» : навчально-методичні матеріали.
- [2] ДСТУ 3008:2015. Інформація та документація. Звіти у сфері науки і техніки. Структура та правила оформлювання. [Чинний від 2017-07-01]. Київ : ДП «УкрНДНЦ», 2016. 31 с.
- [3] PostgreSQL Global Development Group. PostgreSQL 18 Documentation: PREPARE TRANSACTION [Електронний ресурс]. URL: <https://www.postgresql.org/docs/18/sql-prepare-transaction.html> (дата звернення: 28.05.2026).
- [4] PostgreSQL Global Development Group. PostgreSQL 18 Documentation: COMMIT PREPARED [Електронний ресурс]. URL: <https://www.postgresql.org/docs/18/sql-commit-prepared.html> (дата звернення: 28.05.2026).
- [5] PostgreSQL Global Development Group. PostgreSQL 18 Documentation: ROLLBACK PREPARED [Електронний ресурс]. URL: <https://www.postgresql.org/docs/18/sql-rollback-prepared.html> (дата звернення: 28.05.2026).
- [6] PostgreSQL Global Development Group. PostgreSQL 18 Documentation: Transaction Isolation [Електронний ресурс]. URL: <https://www.postgresql.org/docs/18/transaction-iso.html> (дата звернення: 28.05.2026).

ДОДАТОК А

ФАЙЛИ РЕАЛІЗАЦІЇ ТА МАТЕРІАЛИ ПІДТВЕРДЖЕННЯ РЕЗУЛЬТАТІВ

Додаткові умови збереження цілісності даних.

Коректність переказу перевіряється не тільки фактом завершення обох локальних транзакцій, а й виконанням предметних обмежень. Перед переходом до стану PREPARED вузол відправника повинен підтвердити, що рахунок активний і містить достатній залишок, а вузол отримувача — що зарахування дозволене. Додатково координатор має використовувати один глобальний ідентифікатор для всіх локальних операцій, щоб рішення не було застосоване до іншої транзакції.

Таблиця А.1 — Умови коректного завершення банківського переказу

Умова	Перевірка	Результат порушення
Активність рахунку відправника	Поле <code>is_active</code> для рахунку 101 має значення <code>true</code> .	Транзакція не переходить до PREPARED; формується ABORT.
Достатність коштів	Виконується умова <code>balance >= 250.00</code> .	Списання не здійснюється; глобальне рішення — ROLLBACK.
Активність рахунку отримувача	Поле <code>is_active</code> для рахунку 202 має значення <code>true</code> .	Підготовлене списання на Node А скасовується.
Стійкість рішення координатора	Рішення та глобальний ідентифікатор записані до журналу до розсилання завершальних команд.	Після відновлення неможливо надійно визначити потрібну завершальну дію.

За успішного завершення переказу виконується інваріант сукупного залишку: списання суми 250.00 UAH з одного рахунку супроводжується зарахуванням тієї самої суми на інший рахунок. Отже, зміна балансу одного вузла компенсується протилежною зміною другого вузла. Якщо хоча б одна перевірка завершується невдало, жодна з підготовлених змін не має стати остаточною.

Такий порядок перевірок дає змогу розділити технічні відмови та порушення правил предметної області. Помилка мережі або збій координатора потребують відновлення відповідно до журналу рішень, тоді як неактивний рахунок чи недостатній залишок є підставою для прогнозованого відкату операції ще до її фіксації.

Файли та запуск програмного координатора.

Склад файлів практичної реалізації наведено в табл. А.2.

Таблиця А.2 — Файли практичної реалізації

Файл	Призначення
01_setup_nodes.sql	Створює дві бази-вузли, таблиці <code>accounts</code> та <code>transfer_journal</code> , а також початкові баланси.
02_successful_transfer_2pc.sql	Демонструє успішний переказ із використанням <code>PREPARE</code> та <code>COMMIT PREPARED</code> .
03_abort_transfer_2pc.sql	Демонструє відмову <code>Node B</code> та відкат через <code>ROLLBACK PREPARED</code> .
coordinator_2pc.py	Автоматизує роботу координатора 2PC через підключення до двох баз PostgreSQL.
requirements.txt	Визначає залежність Python-драйвера <code>psycopg</code> для запуску координатора.

Лістинг А.1 — Запуск автоматизованого координатора

```
1 python -m pip install -r requirements.txt
2 python coordinator_2pc.py 250.00
```

Програма застосовує два підключення з режимом `autocommit=True`, оскільки команди `COMMIT PREPARED` та `ROLLBACK PREPARED` виконуються поза іншим транзакційним блоком. Перед передаванням команди `COMMIT` рішення записується у файл `coordinator_decisions.log`; це дає змогу після збою повторити завершення невирішеної `prepared transaction`.

Перелік ілюстрацій для звіту.

Для підтвердження виконання практичної частини до звіту слід додати скріншоти: початкових балансів після виконання `01_setup_nodes.sql`; стану `pg_prepared_xacts` після фази `PREPARE`; кінцевих балансів після `COMMIT PREPARED`; результату відкату `ROLLBACK`; консольного виводу `coordinator_2pc.py` із рішенням координатора.